

```
> execute_system_one()
```

base64: aW50ZWxsaWdlbmN1

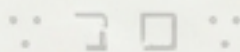


The New Intelligence Primitive

Intelligence beyond chat. Introducing System One Models and Jev.

base64: aW50ZWxsaWdlbmN1

base64: aW50ZWxsaWdlbmN1

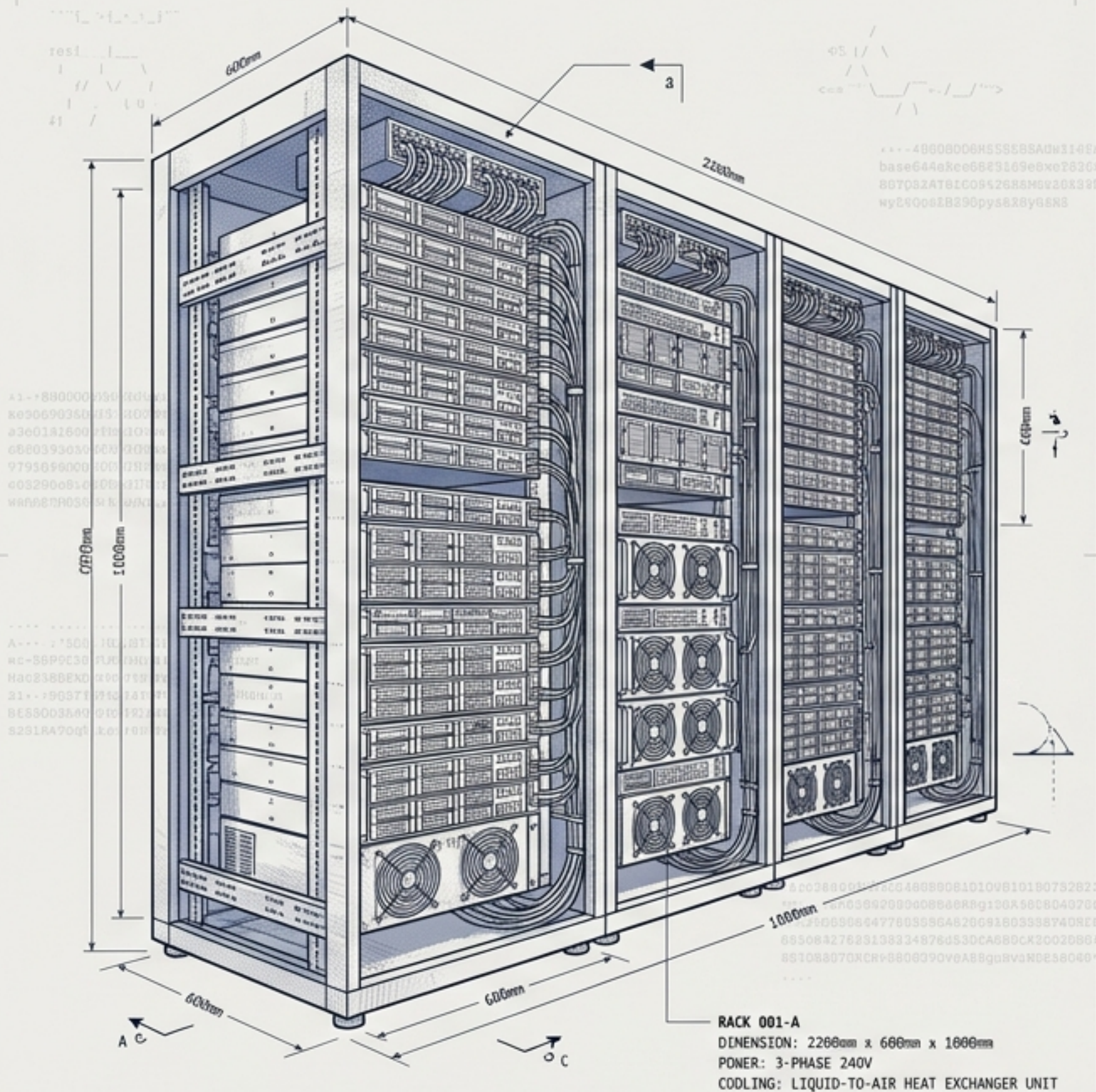


data:application/pdf;base64.aW50ZWxsaWdlbmN1

data:application/pdf;base64.aW50ZWxsaWdlbmN1



data:application/pdf;base64...



Compute drives progress, but only if you optimize for the right task.

Current language models possess immense raw capability. Yet, their **fundamental optimization target is flawed for software automation.**

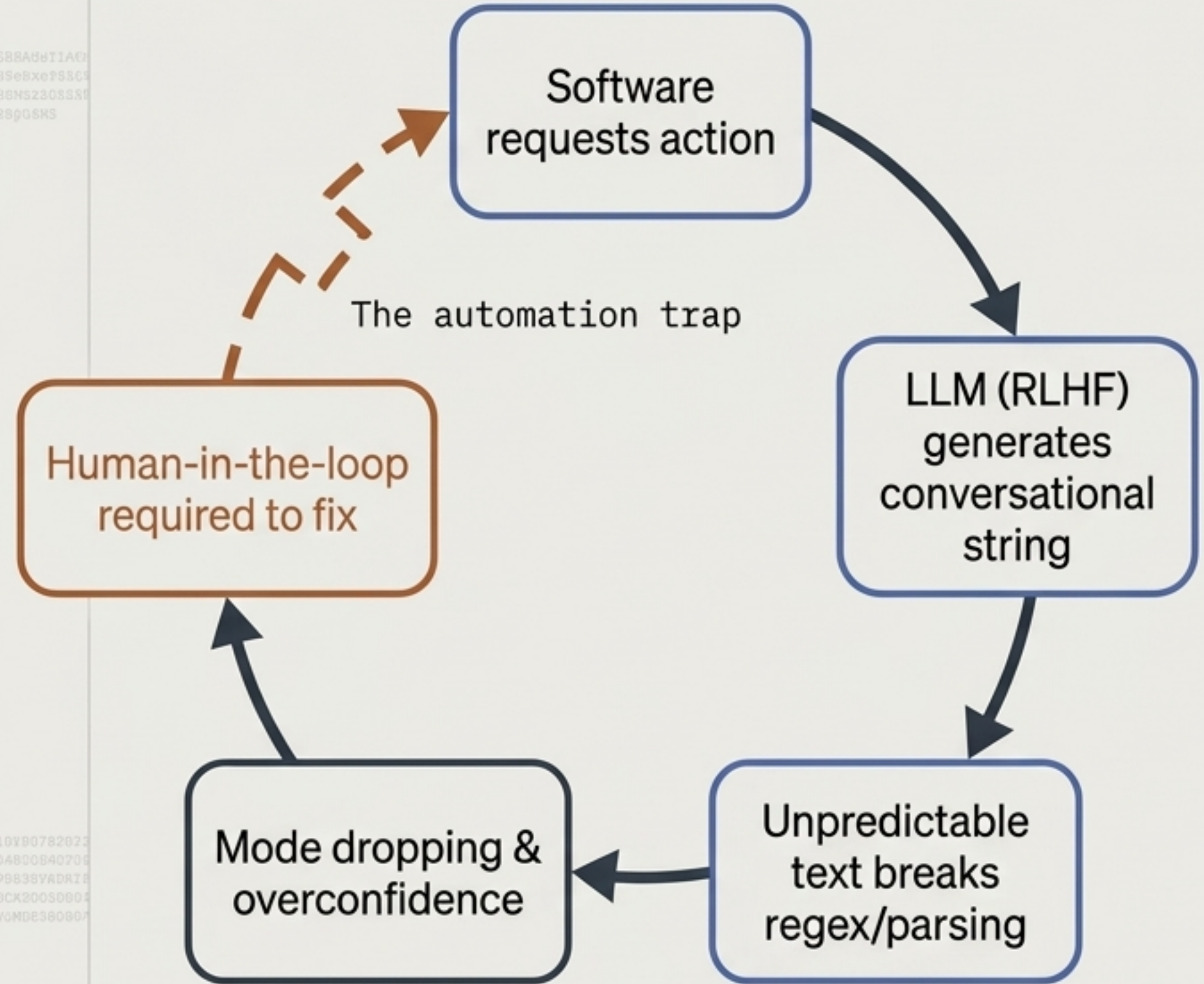
We are directing unimaginable compute power toward the wrong objective.

What good is progress if you are optimizing for human conversation?

RLHF models produce words to please people.

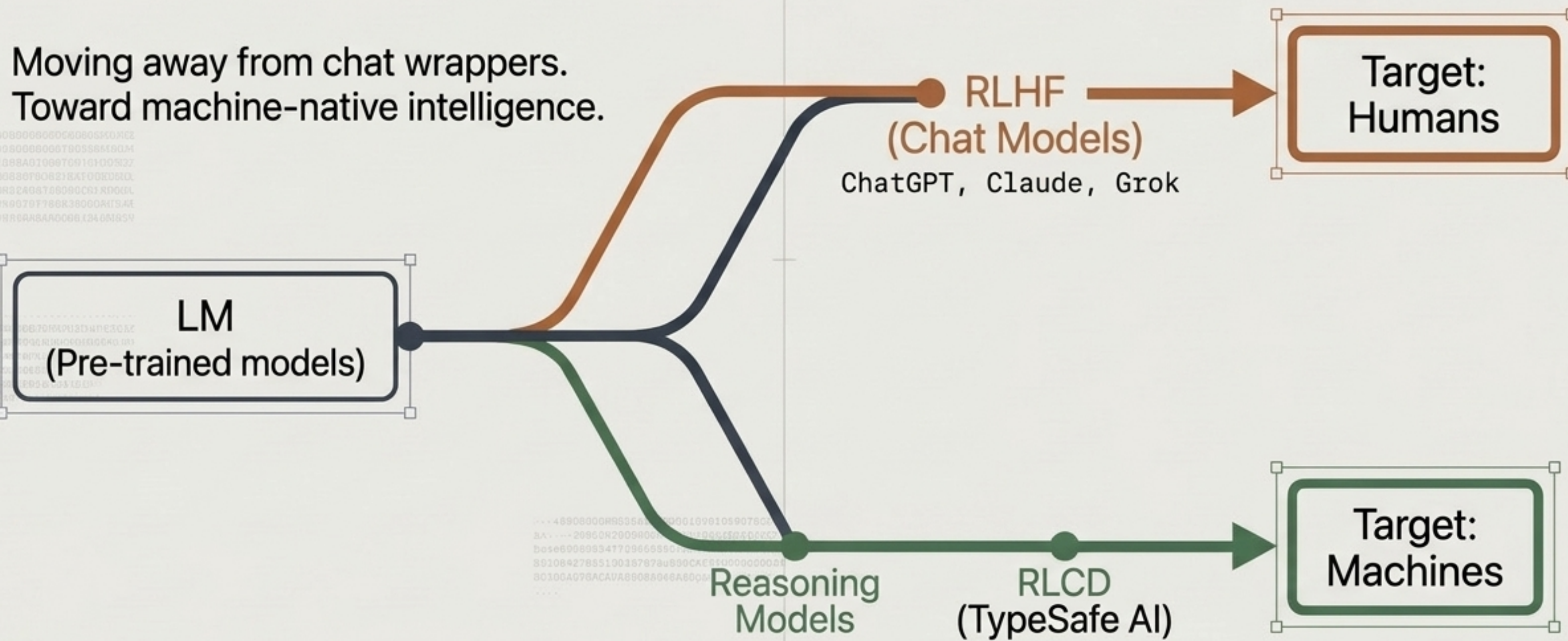
Reinforcement Learning from Human Feedback (RLHF) creates models that are superhuman at instruction following.

They are optimized to assist humans, not to make reliable, autonomous decisions inside software.



We took the opposite research direction.

Moving away from chat wrappers.
Toward machine-native intelligence.

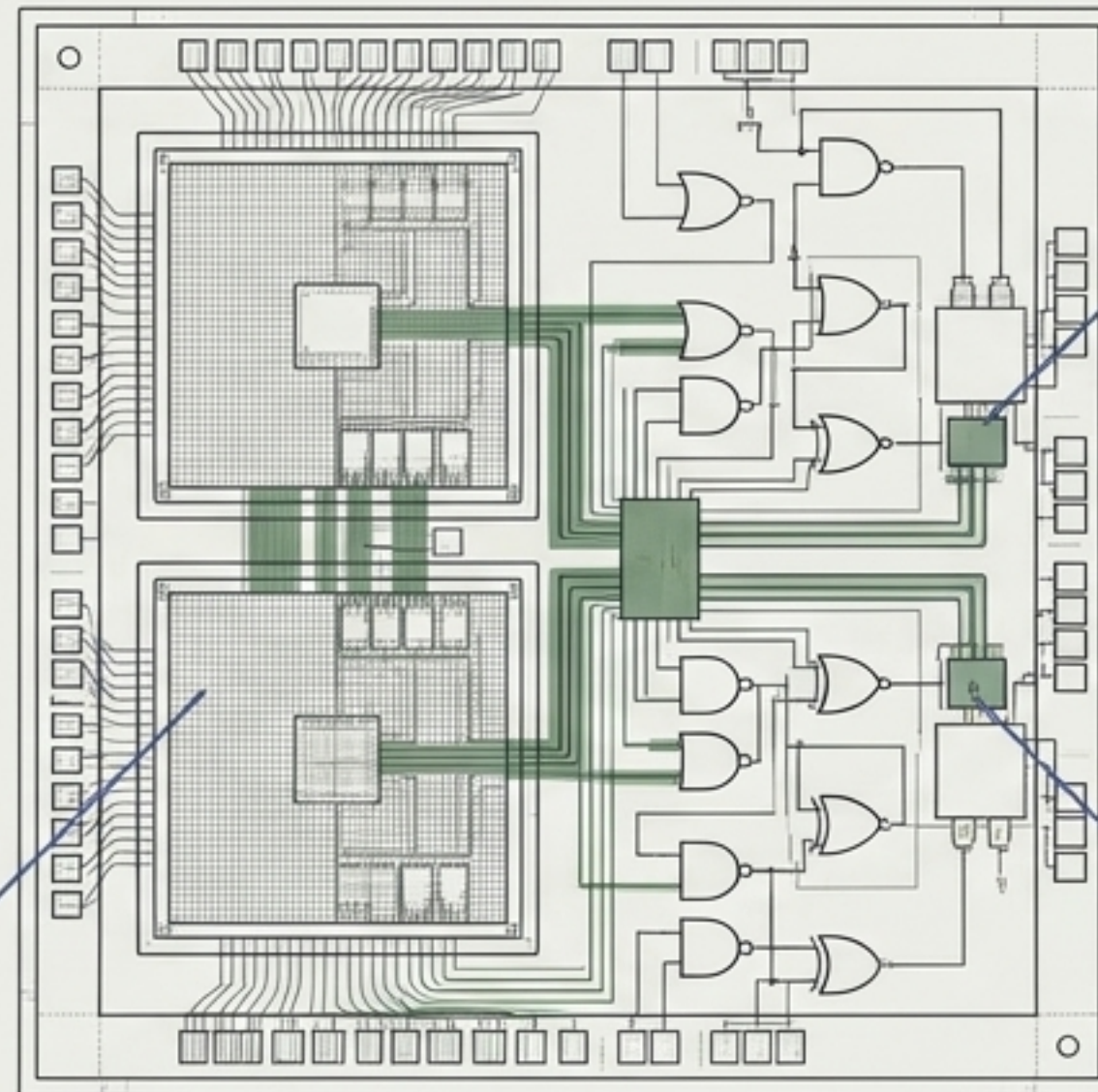


Software requires code, not conversational text.

	Legacy LLMs	System One
Primary User	Humans	Machines
Training Method	RLHF	RLCD
Output Format	Strings & Text	Typed Decisions
Reliability	Overconfident / Hallucinates	Calibrated / Self-consistent
Latency	Seconds	Milliseconds

Introducing Jev: The first System One Model.

Built with a new architecture, a new sampler, and a new training algorithm:
Reinforcement Learning for Calibrated Decisions (RLCD).



Version: 0.01

Trait: Gives AI the
properties of code.

Status: Optimized
for Automation

Decisions, not strings.

Jev produces typed decisions that are reliable, fast, and type-safe. Software cannot confidently parse poetry; it needs types.

Legacy parsing

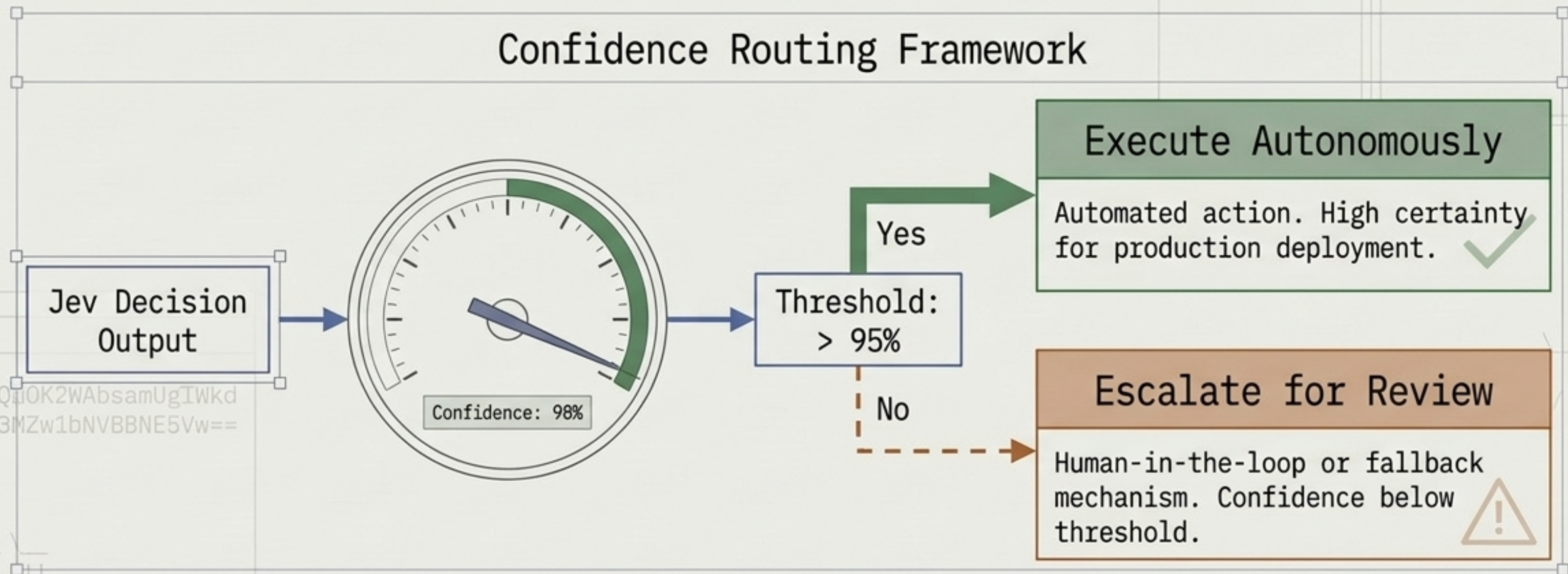
```
legacy_parser.py × — □ ×  
1 import re  
2  
2 output = llm.generate_text(prompt)  
3 match = re.search(r'```json\n(.*)\n```', output)  
4 try:  
5     data = json.loads(match.group(1))  
6 except ValueError:  
7     trigger_human_fallback()
```

Type-safe native execution

```
jev_decision.jev × — □ ×  
1 import jev  
2  
2 Decision<Route, 0.95> output = jev.execute(context);  
3 system.route(output);
```

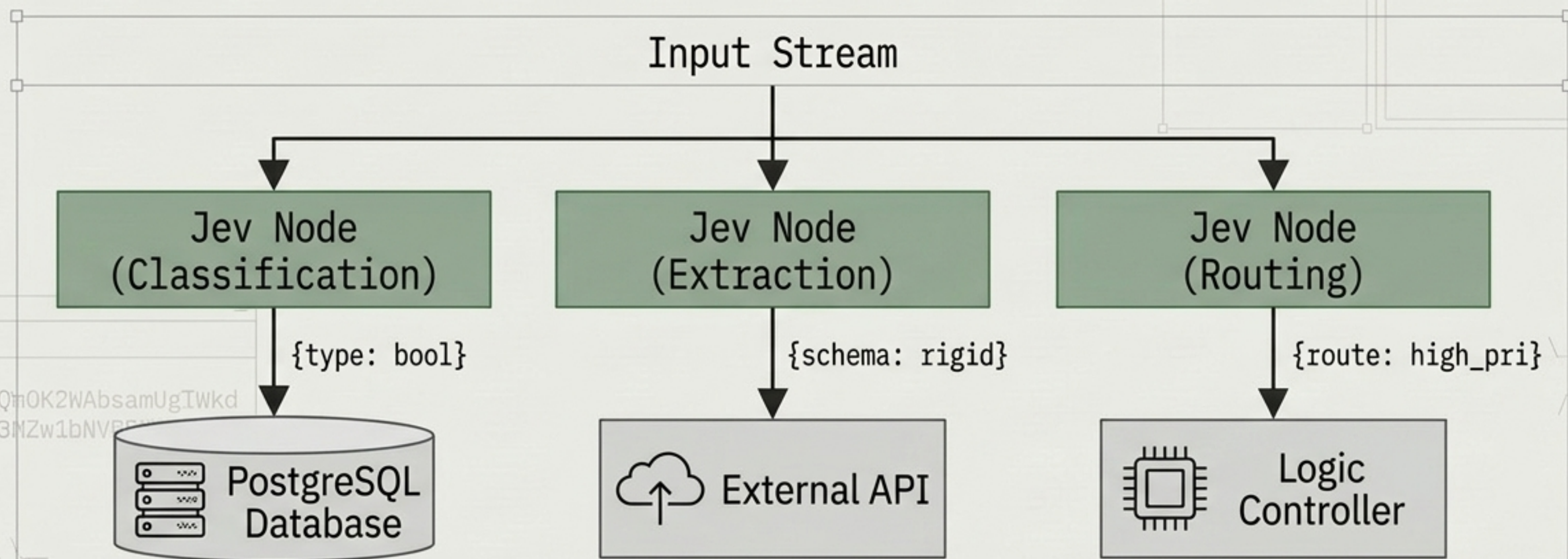
The mathematical eradication of hallucinations.

Every Jev decision includes a calibrated estimate of confidence.
Set your thresholds so your software accounts for uncertainty natively.



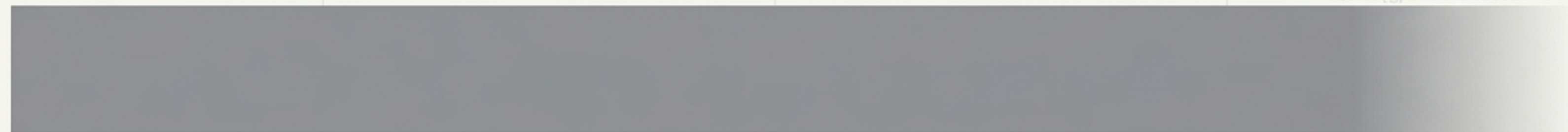
Build larger workflows with total control.

Combine calibrated decisions in code. Maintain absolute control over how machine-native intelligence is routed, used, and audited within your architecture.



Built for automation requires **sub-second** latency.

Legacy LLMs - 8.566s



193.6x Faster

Jev - 0.114s



*Based on workflows for System One tasks.

Economics that rewrite software architecture.

Jev's intelligence per dollar is literally off the charts.

444.6x Cheaper

Cost per task (LLMs): \$0.013880

Cost per task (Jev): **\$0.000081**

\$42 Per Billion input tokens.

238x lower input price than Claude Fable 5.1.

The machine-native intelligence frontier.

We are not competing with chat models.
We have opened an entirely new quadrant for autonomous software.



Workflow Intelligence

Bloated Chat Models

Jev (System One)

Cost & Latency

System One: System Diagnostics

> Is Jev just a smaller LLM?

No. It uses a fundamentally new architecture and training algorithm (RLCD) optimized for automation.

> How is this different from JSON mode?

JSON mode forces a language model to guess syntax. Jev outputs native typed decisions with mathematically calibrated probabilities.

> Is Jev deterministic?

It relies on calibrated probabilities, giving your code the exact confidence scores needed to handle uncertainty deterministically.

The waitlist is dead. Come build with us.

```
float Q_rsqrt( float number ) {  
    long i;  
    float x2, y;  
    const float threehalfs = 1.5F;  
    x2 = number * 0.5F;  
    y = number;  
    i = * ( long * ) &y;  
    i = 0x5f3759df - ( i >> 1 );  
    y = * ( float * ) &i;  
    y = y * ( threehalfs - ( x2 * y * y ) );  
    return y;  
}
```

> create_account()

[Console]

[Discord]

[Open Roles]